



ALLIANCE
UNIVERSITY

*Private University established in Karnataka State by Act No. 84 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi*

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Comparison of Deep Learning

Architectures

Project 3 Report Course: Deep Learning

Submitted by: Hamzah Tareq (2411021240077) Date: 31
August 2026

1. Project Overview

Not every kind of data suits the same kind of neural network. A spreadsheet of customer records, a photograph, and a sentence of text all carry information in very different ways, and the architecture used to process them has to be built around that difference. This report looks at three foundational architectures — Feedforward Neural Networks (FNNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs) — and examines how each one is structured, what kind of input it expects, and where it is actually used in practice.

The report is organised around the three tasks set out in the project brief: studying the basic architecture of each network, identifying suitable real-world applications, and comparing the three side by side on structure, input data, and use cases.

2. Task 1 & 2: Feedforward Neural Networks (FNN)

2.1 Architecture

A feedforward network is the simplest and oldest neural network design, often called a Multilayer Perceptron (MLP). Information travels in a single direction — from the input layer, through one or more hidden layers, to the output layer — and there is no path for information to move backward or sideways once it has been processed. Every neuron in one layer is typically connected to every neuron in the next, which is why these are described as fully connected layers.

Each neuron performs two simple steps: it multiplies its inputs by learned weights and adds them together with a bias term, then passes that sum through a non-linear activation function such as ReLU or Sigmoid. Stacking many such neurons across several layers allows the network to approximate fairly complex functions, even though each individual neuron's operation is simple.

2.2 Input Data

FNNs expect a fixed-length vector of numbers as input. There is no built-in concept of position or neighbourhood — if an image is fed into an FNN,

it first has to be flattened into a single long list of pixel values, which throws away information about which pixels were next to each other. This makes FNNs a poor fit for images or sequences, but a strong fit for tabular data where each column is already an independent feature.

2.3 Applications

- Credit scoring and loan default prediction from applicant financial records
- Customer churn prediction using account activity and demographic features
- House price estimation from structured attributes such as area, location, and age

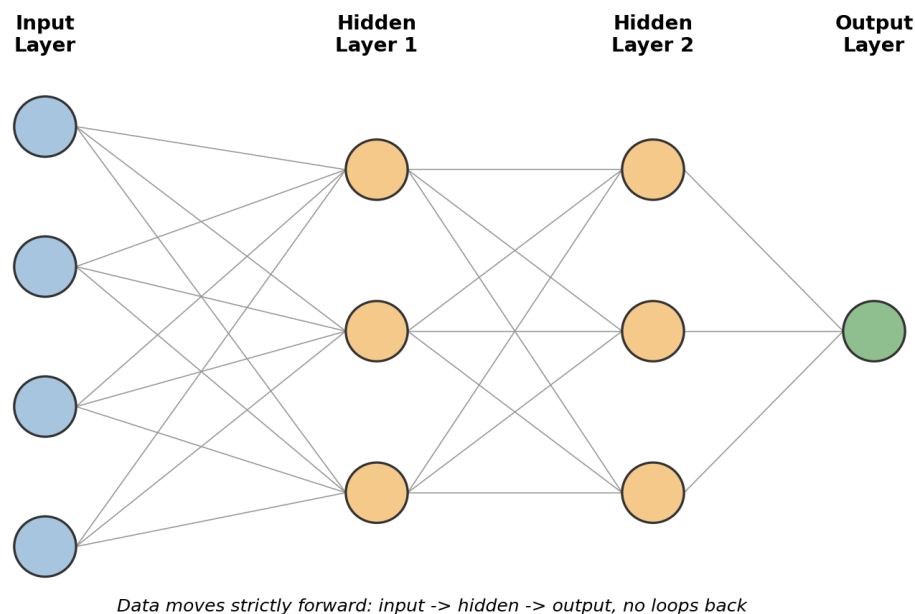


Figure 1: A feedforward network — data flows one way, layer by layer, with no memory of previous inputs.

3. Task 1 & 2: Convolutional Neural Networks (CNN)

3.1 Architecture

CNNs were built specifically to work with grid-shaped data such as images. Rather than connecting every input pixel to every neuron, a CNN slides a small set of learnable filters across the image. Each filter looks at a small local patch at a time

and produces a single value summarizing what it found there, building up a 'feature map' as it moves across the whole image. Because the same filter is reused at every position, the network needs far fewer parameters than a fully connected layer would, and it can recognize a pattern no matter where in the image it appears.

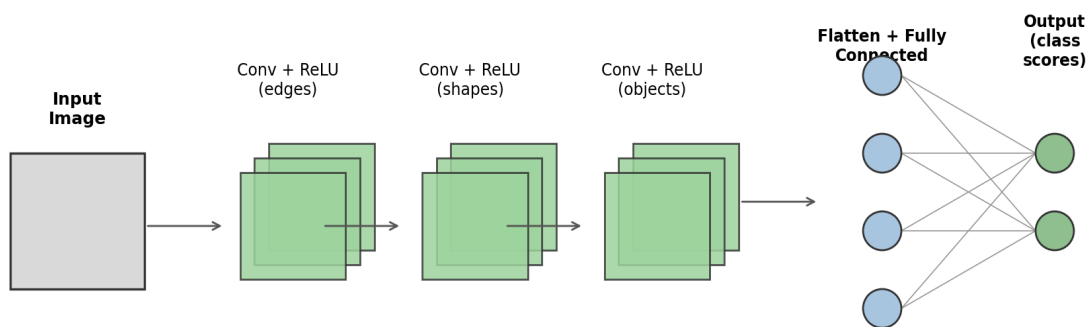
A typical CNN stacks several of these convolutional layers. Early layers tend to pick out simple patterns like edges and colour contrasts; deeper layers combine these into more complex shapes, textures, and eventually whole objects. Pooling layers are usually inserted between convolutions to shrink the feature maps and make the network less sensitive to small shifts in position. After several rounds of convolution and pooling, the resulting features are flattened and passed to one or more fully connected layers that produce the final prediction.

3.2 Input Data

CNNs take multi-dimensional input directly — for example, an RGB image is fed in as a width x height x 3-channel tensor, with no flattening required. Keeping this grid shape intact is exactly what lets the convolution operation exploit the fact that nearby pixels are related to one another.

3.3 Applications

- Face recognition and unlock systems on smartphones
- Object and lane detection for self-driving vehicles
- Detecting tumours or fractures in X-ray and MRI scans



Small filters slide across the image to build feature maps, which are then classified

Figure 2: A CNN — filters slide across the image to build feature maps before a final classification step.

4. Task 1 & 2: Recurrent Neural Networks (RNN)

4.1 Architecture

Neither FNNs nor CNNs have any sense of order between separate inputs — each input is treated on its own. That breaks down for data like sentences, speech, or stock prices, where what came before directly affects what comes next. RNNs solve this by adding a feedback connection: at every time step, the hidden state is calculated from both the current input and the hidden state from the previous step. This hidden state acts like a running summary of everything the network has seen so far in the sequence.

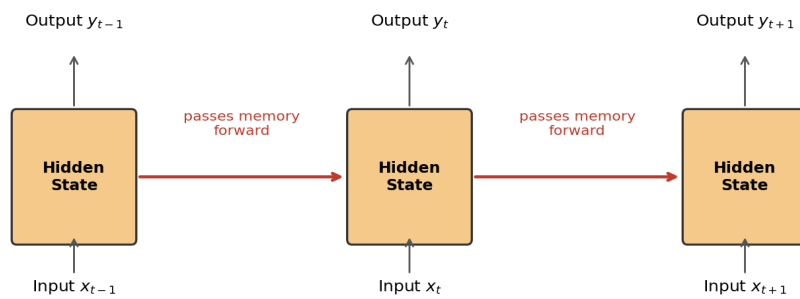
Because the same weight matrices are applied at every time step, an RNN can process sequences of any length without needing new parameters for each position. In practice, plain RNNs struggle to retain information over very long sequences — a problem known as vanishing (or exploding) gradients — which is why variants such as LSTM and GRU are commonly used instead, though the core recurrent idea stays the same.

4.2 Input Data

RNNs are built for ordered sequences: words in a sentence, audio samples over time, or daily readings in a time series. The order of the data changes its meaning, and the network is explicitly designed to take that ordering into account.

4.3 Applications

- Machine translation and text generation
- Speech-to-text transcription for voice assistants
- Forecasting stock prices or other time-series trends



The same weights are reused at every time step, and each hidden state carries information from earlier steps

Figure 3: An RNN — the hidden state is passed forward from one time step to the next, giving the network a form of memory.

5. Task 3: Comparative Analysis

The table below summarises the three architectures side by side across structure, input data, and typical applications, as required by Task 3 of the project.

Aspect	Feedforward Network (FNN)	Convolutional Network (CNN)	Recurrent Network (RNN)
Connection pattern	Every neuron connects to all neurons in the next layer	Small filters connect to local patches of the input	Neurons connect forward in time and to themselves via a hidden state
Handles sequence/order?	No — inputs are treated independently	No — spatial layout matters, not time	Yes — designed specifically for ordered data
Handles spatial structure?	No — inputs must be flattened	Yes — this is its main strength	Not directly
Typical input	Fixed-length feature vectors (tabular data)	Images, grids, video frames	Sequences: text, audio, time series
Parameter sharing	None	Same filter reused across the image	Same weights reused across every time step
Main weakness	Cannot exploit spatial or temporal patterns	Computationally heavy; no memory of order	Struggles with very long sequences (vanishing gradients)
Typical use case	Loan approval, price prediction	Image classification, medical scans	Language modeling, speech recognition

6. Conclusion

The three architectures are best understood as answers to three different questions about the shape of the input data. FNNs assume each input feature is independent and are well suited to tabular data. CNNs assume nearby pixels are related and are built to exploit spatial structure in images. RNNs assume that order matters and are built to carry information forward through a sequence. Choosing the right architecture starts with recognising which of these assumptions actually matches the data at hand — using an FNN on raw images, or a CNN on a sentence, would throw away the exact structure the better-suited architecture is designed to capture.